Documentation technique

Plateforme de réservation (salles, restaurants, hôtels)

Projet de stage : semaine 1, cadrage et mise en place

CakePHP 5.4, PHP 8.2, MySQL, Git et GitHub

Auteur : [Ton nom]

Date : 31 octobre 2026

Dépôt : https://github.com/fleurssenahounvo-has/reservation-app

1. Introduction

Ce document retrace pas à pas tout ce qui a été fait pendant la première semaine du projet : le cadrage, l'installation de l'environnement de développement, la création du dépôt GitHub, la configuration de la base de données et la création des tables par migrations. Chaque étape contient les commandes exactes à copier et, quand c'est utile, une capture d'écran.

Il sert aussi de base pour le rapport de stage et permet de refaire l'installation sur une autre machine.

1.1 Le problème

La réservation traditionnelle est lente et manque d'information. Un client doit souvent se déplacer ou appeler pour apprendre que la salle, la table ou la chambre est déjà occupée. L'application permet à toute personne de consulter les disponibilités à distance et de réserver sans compte obligatoire. Elle aide aussi les établissements à gérer leurs réservations sans conflit.

1.2 Objectifs

- Réduire le temps nécessaire pour réserver.
- Supprimer les déplacements inutiles grâce à des disponibilités toujours exactes.
- Éviter les doubles réservations sur une même ressource et une même période.

1.3 Périmètre de la version 1

- Types d'établissements : salles (conférence, réception) et restaurants. Les hôtels et auberges viendront ensuite, avec le même moteur.
- Aucun compte n'est exigé pour réserver : le client laisse ses coordonnées directement dans la réservation.
- Trois acteurs : client, gestionnaire d'établissement, administrateur de la plateforme.

1.4 Technologies choisies

Couche	Choix et raison
Interface	HTML, CSS, JavaScript (déjà maîtrisés)
Serveur	PHP 8.2 avec CakePHP 5.4 : framework MVC maintenu, avec ORM, migrations et tests intégrés. FuelPHP a été écarté car il n'est plus guère maintenu.
Base de données	MySQL (MariaDB fournie par XAMPP) : transactions et verrous nécessaires pour éviter les conflits.
Versionnage	Git et GitHub (dépôt public, vitrine pour le stage)
Tests	PHPUnit (prévu en semaine 6)

2. Cadrage fonctionnel

2.1 Diagramme de cas d'utilisation

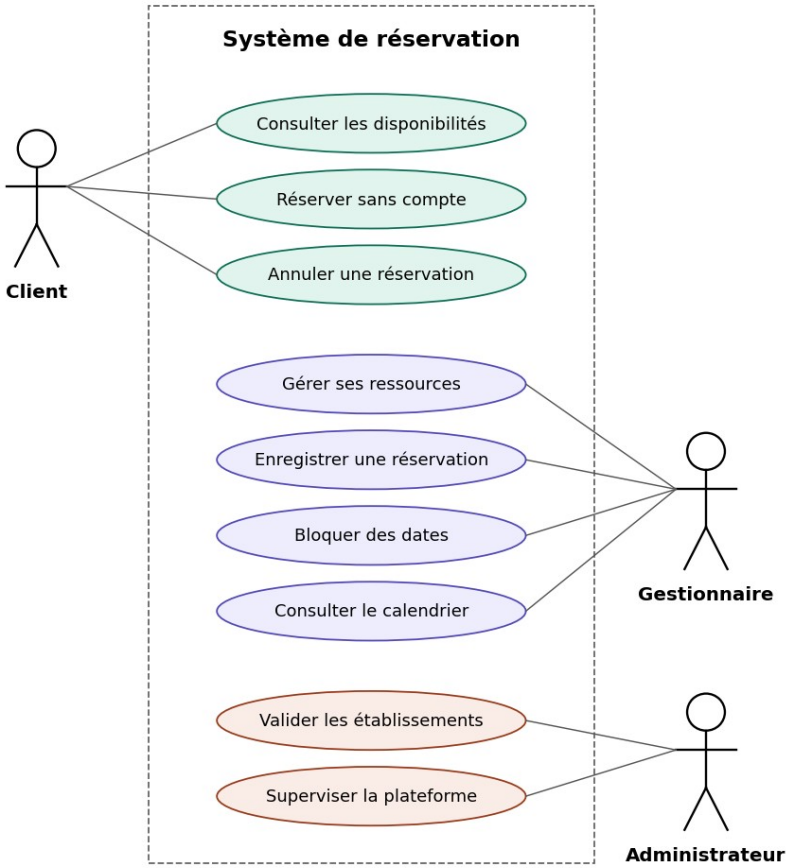


Figure 1 : cas d'utilisation du système de réservation

2.2 User stories

1. En tant que client, je veux voir les créneaux libres d'une salle ou d'une table afin d'éviter un déplacement inutile.
2. En tant que client, je veux réserver en ligne afin de gagner du temps.
3. En tant que client, je veux recevoir une confirmation afin d'être sûr que ma place est garantie.
4. En tant que gestionnaire, je veux enregistrer une réservation faite par téléphone afin que le calendrier reste à jour.
5. En tant que gestionnaire, je veux bloquer des dates (travaux, fermeture) afin qu'aucune réservation ne soit acceptée.
6. En tant que client, je veux réserver sans créer de compte afin de gagner du temps.
7. En tant que client, je veux annuler ma réservation afin de libérer le créneau si je ne peux plus venir.
8. En tant que gestionnaire, je veux ajouter mes salles, tables ou chambres avec leur capacité et leur prix afin que les clients les voient.
9. En tant que gestionnaire, je veux voir le calendrier de mes réservations afin de préparer l'accueil.
10. En tant qu'administrateur, je veux valider les établissements inscrits afin d'éviter les faux établissements.

2.3 Classement des fonctionnalités

Catégorie	Fonctionnalités
Indispensables	Liste des ressources, calendrier des disponibilités, réservation sans compte, détection des conflits, annulation, ajout d'une réservation par le gestionnaire (téléphone, comptoir), blocage de dates
Utiles	Connexion du gestionnaire, notifications par e-mail ou SMS, historique des réservations, expiration automatique des réservations en attente
Bonus	Paiement simulé, réservations récurrentes, statistiques d'occupation, comptes clients

2.4 Règles de gestion principales

- Une ressource ne peut pas avoir deux réservations actives qui se chevauchent. Deux périodes se chevauchent si `debut1 < fin2` et `debut2 < fin1`.
- Une période bloquée (fermeture, travaux) refuse toute réservation.
- Statuts d'une réservation : `en_attente`, `confirmee`, `annulee`, `expiree`.
- Sources d'une réservation : `en_ligne`, `telephone`, `comptoir`.
- En cas de demandes simultanées, une transaction avec verrouillage de la ressource garantit qu'une seule réservation passe (`SELECT ... FOR UPDATE`).

3. Installation de l'environnement

Poste utilisé : Windows, avec XAMPP (Apache, MySQL, PHP) et PowerShell.

3.1 Installer Composer

Composer est le gestionnaire de paquets de PHP. Il est téléchargé sur getcomposer.org. Pendant l'installation, l'écran « Proxy Settings » apparaît : il faut décocher la case « Use a proxy server to connect to internet », sauf si le réseau impose un proxy. Laissée cochée avec un champ vide, elle risque de provoquer des erreurs de connexion.

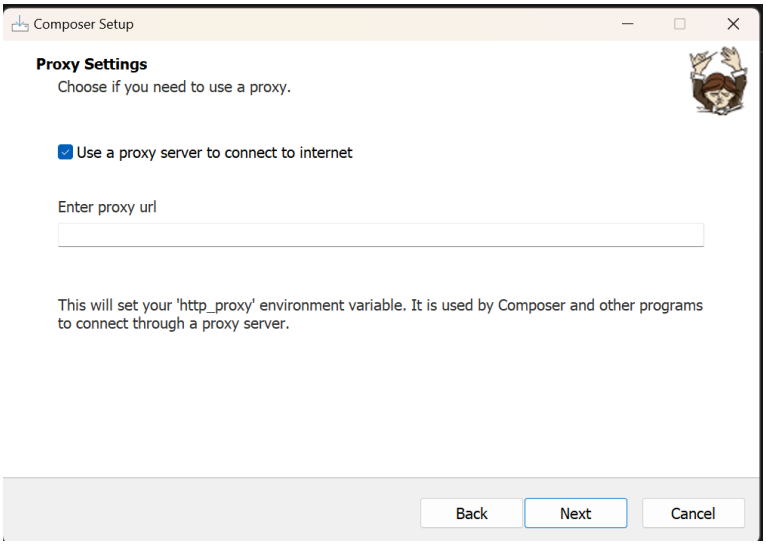


Figure 2 : écran Proxy Settings de l'installateur Composer (case à décocher)

3.2 Vérifier les versions

Après l'installation, fermer puis rouvrir le terminal, et vérifier :

Terminal

```
composer -V
php -v
```

Résultat obtenu : Composer 2.11.1 et PHP 8.2.12. CakePHP 5 demande PHP 8.1 ou plus : la version convient. Attention : `composer -v` (v minuscule) affiche l'aide complète ; la version seule s'obtient avec `-V` majuscule.

3.3 Activer les extensions PHP nécessaires

La commande `php -m` liste les extensions chargées. Dans la liste obtenue, `mbstring`, `openssl` et `pdo_mysql` étaient présentes, mais pas `intl`, que CakePHP exige. L'extension `zip` manquait aussi, ce qui a fait échouer la première installation de CakePHP avec le message « The zip extension and unzip/7z commands are both missing ».

1. Trouver le fichier php.ini réellement utilisé : `php --ini` (ligne « Loaded Configuration File »). Ici : `C:\xampp\php\php.ini`.
2. L'ouvrir : `notepad C:\xampp\php\php.ini`.
3. Chercher (Ctrl + F) les lignes `;extension=intl` et `;extension=zip`, puis supprimer le point-virgule au début pour obtenir `extension=intl` et `extension=zip`.
4. Enregistrer, fermer et rouvrir le terminal.
5. Vérifier : `php -m | findstr intl` et `php -m | findstr zip`. Chaque mot doit s'afficher.

4. Installation de CakePHP

Terminal (PowerShell)

```
cd C:\xampp\htdocs
composer create-project --prefer-dist cakephp/app reservation-app
```

À la question « Set Folder Permissions ? », répondre Y. Version installée : CakePHP 5.4.0 au téléchargement, affichée 5.4.3 dans la console.

Si une tentative échoue et laisse un dossier incomplet, le supprimer avant de relancer :

Terminal

```
Remove-Item -Recurse -Force C:\xampp\htdocs\reservation-app
```

4.1 Lancer le serveur de développement

Terminal

```
cd reservation-app
bin\cake server
```

Le terminal affiche `php server is running at http://localhost:8765/`. Ce terminal doit rester ouvert : s'il est fermé, le serveur s'arrête. Pour taper d'autres commandes, ouvrir un deuxième terminal.

Tant que le serveur n'est pas lancé (ou que le projet n'est pas installé), le navigateur affiche l'erreur ERR_CONNECTION_REFUSED :

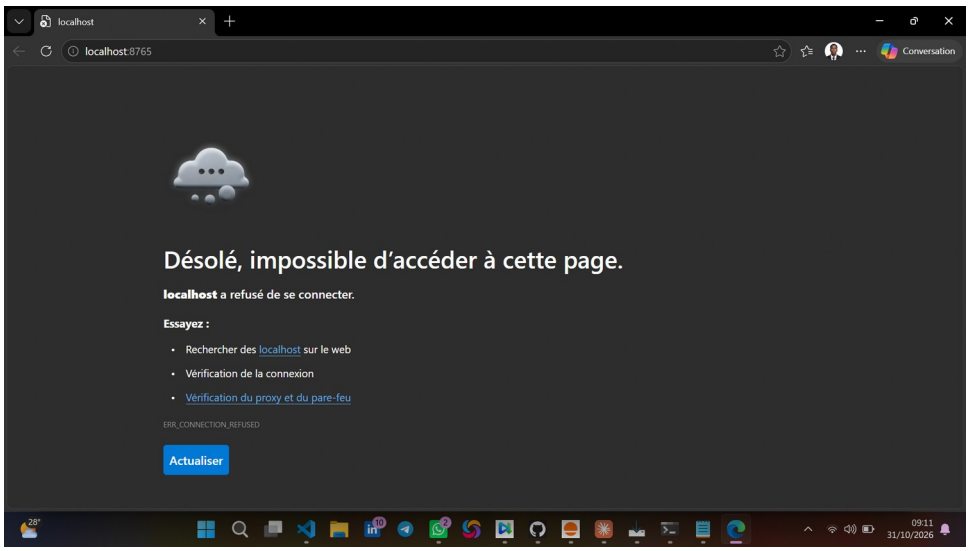


Figure 3 : erreur ERR_CONNECTION_REFUSED quand le serveur n'est pas lancé

Une fois le serveur lancé, la page d'accueil de CakePHP s'affiche à l'adresse http://localhost:8765. Toutes les vérifications de l'environnement sont au vert (PHP, mbstring, openssl, intl, dossiers tmp et logs).

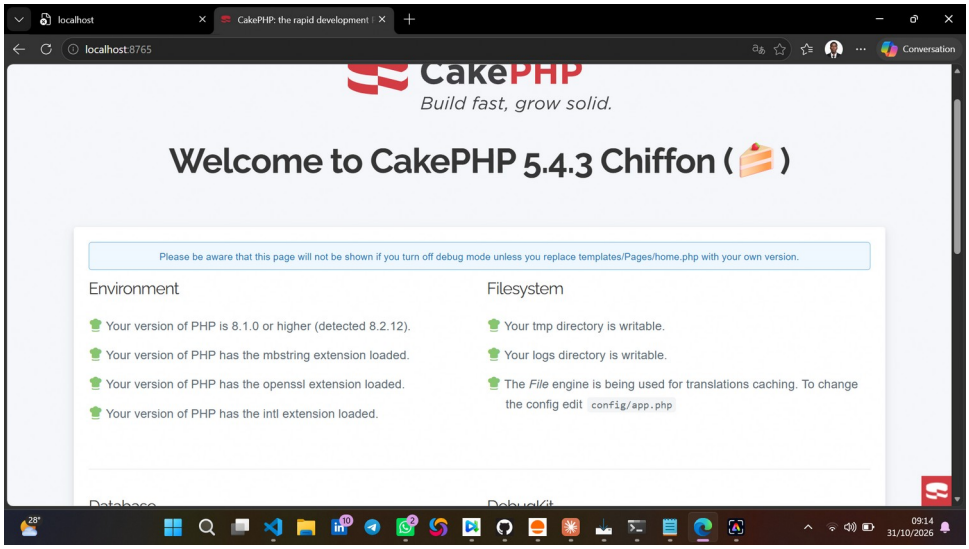


Figure 4 : page d'accueil de CakePHP 5.4.3, environnement valide

5. Git et GitHub

5.1 Création du dépôt

Dépôt public nommé `reservation-app`. Public, car il sert de vitrine pour le stage. Il ne doit jamais contenir de mots de passe ni de vraies données : le fichier `config/app_local.php`, qui porte les identifiants de base de données, est exclu par le `.gitignore` fourni avec CakePHP.

5.2 Initialisation locale

Se présenter à Git (une seule fois)

```
git config --global user.name "Ton Nom"
git config --global user.email "ton-email@exemple.com"
```

Dans C:\xampp\htdocs\reservation-app

```
git init -b main
git status
```

```
git add .
git commit -m "Initial commit : installation de CakePHP 5.4"
```

5.3 Relier au dépôt GitHub et résoudre le conflit

Terminal

```
git remote add origin https://github.com/fleurssenahounvo-has/reservation-app.git
git push -u origin main
```

Le premier `push` a été refusé (« rejected, fetch first ») : le dépôt GitHub contenait déjà un README et un .gitignore créés à la création du dépôt. Il a fallu fusionner :

Terminal

```
git pull origin main --allow-unrelated-histories --no-rebase
# conflit sur .gitignore et README.md :
git checkout --ours .gitignore      # garder la version CakePHP
git add .gitignore
git checkout --theirs README.md     # garder le README de GitHub
git add README.md
git commit -m "Fusion avec le dépôt distant"
git push -u origin main
```

Vérification sur GitHub : pas de dossier `vendor`, pas de fichier `config/app_local.php`. Seul `app_local.example.php` est présent.

Contrôle depuis le terminal

```
git ls-files config
```

6. Configuration de la base de données

6.1 Créer la base

1. Dans XAMPP Control Panel, démarrer MySQL (et Apache pour phpMyAdmin).
2. Ouvrir http://localhost/phpmyadmin.
3. Nouvelle base de données : nom `reservation_db`, interclassement `utf8mb4_unicode_ci`, puis Créer.

6.2 Configurer CakePHP

Le fichier à modifier est `config/app_local.php` (et non `app_local.example.php`, qui n'est qu'un modèle). S'il n'existe pas, le créer par copie :

Terminal

```
copy config\app_local.example.php config\app_local.php
```

Dans la section `Datasources` puis `'default'`, modifier trois lignes (autour des lignes 50 à 53) :

config/app_local.php

```
'username' => 'root',
'password' => '',
'database' => 'reservation_db',
```

La section `'test'` plus bas ne se modifie pas pour l'instant : elle servira aux tests PHPUnit de la semaine 6. L'utilisateur `root` sans mot de passe est la valeur par défaut de XAMPP et n'est acceptable qu'en local.

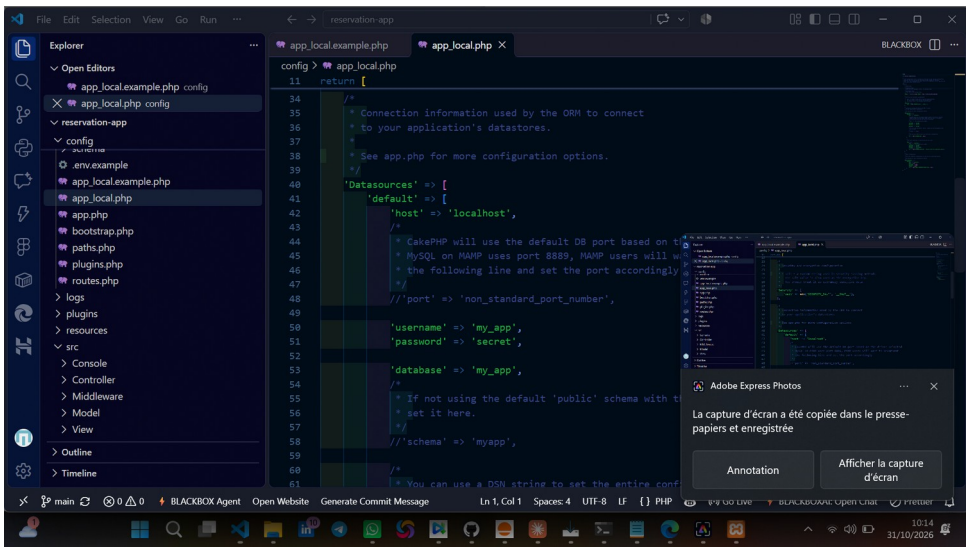


Figure 5 : fichier config/app_local.php ouvert dans VS Code (section default, lignes 50 à 53 à modifier)

6.3 Tester la connexion

En rafraîchissant http://localhost:8765, la section Database de la page d'accueil indique « CakePHP is able to connect to the database » et « DebugKit can connect to the database ».

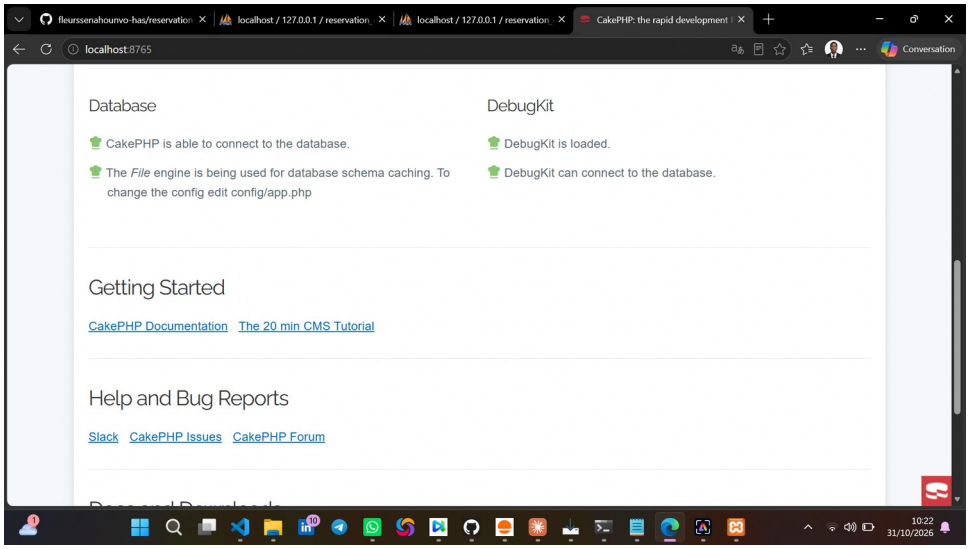


Figure 6 : connexion à la base de données réussie

7. Création des tables par migrations

Une migration est un fichier PHP qui décrit une table. Elle est versionnée dans Git, ce qui permet de recréer la base sur n'importe quelle machine. Le principe est identique pour chaque table :

Cycle de travail

```
bin\cake bake migration NomDeLaMigration champ:type ...   # génère le fichier
# modifier le fichier dans config/Migrations, puis :
bin\cake migrations migrate                               # crée la table
bin\cake migrations status                                # vérifie (up = appliquée)
git add config/Migrations
git commit -m "Migration : ..."
git push
```

Pour annuler la dernière migration (et supprimer sa table) pendant le développement : `bin\cake migrations rollback`. Dans cette installation, le registre des migrations est la table `cake_migrations`.

7.1 Table etablissements

Génération

```
bin\cake bake migration CreateEtablissements nom:string[150] type:string[30]
adresse:string[255] telephone:string[30]
```

La version générée rendait toutes les colonnes obligatoires et n'avait pas de dates. Elle a été corrigée : adresse et téléphone facultatifs, ajout de `created` et `modified`.

config/Migrations/..._CreateEtablissements.php (méthode change)

```
public function change(): void
{
    $table = $this->table('etablissements');
    $table->addColumn('nom', 'string', ['limit' => 150, 'null' => false])
          ->addColumn('type', 'string', ['limit' => 30, 'null' => false])
          ->addColumn('adresse', 'string', ['limit' => 255, 'null' => true])
          ->addColumn('telephone', 'string', ['limit' => 30, 'null' => true])
          ->addColumn('created', 'datetime', ['null' => true])
          ->addColumn('modified', 'datetime', ['null' => true])
          ->create();
}
```

Après `migrate`, phpMyAdmin affiche la table `etablissements` (vide) à côté de `cake_migrations` :

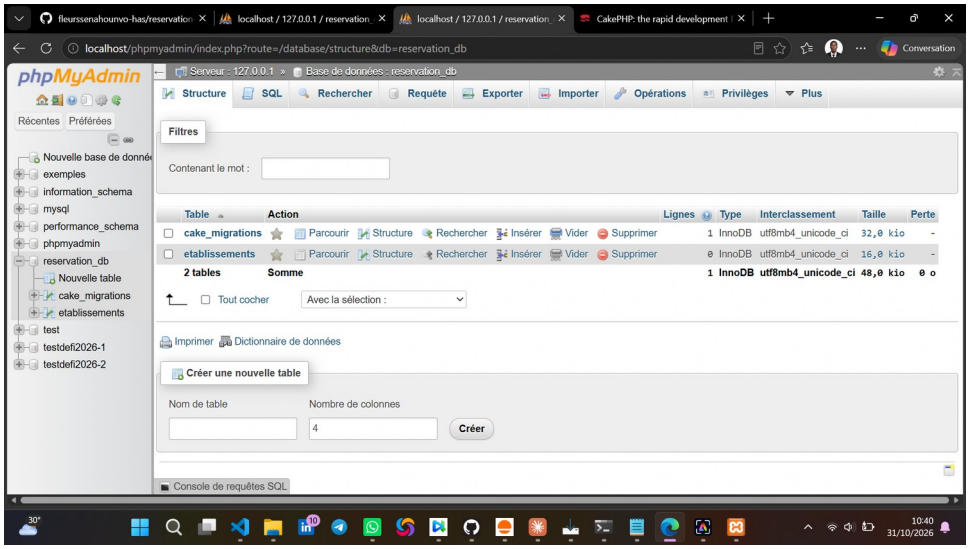


Figure 7 : base reservation_db avec les tables cake_migrations et etablissements

La requête `SELECT * FROM etablissements` confirme les 7 colonnes attendues :

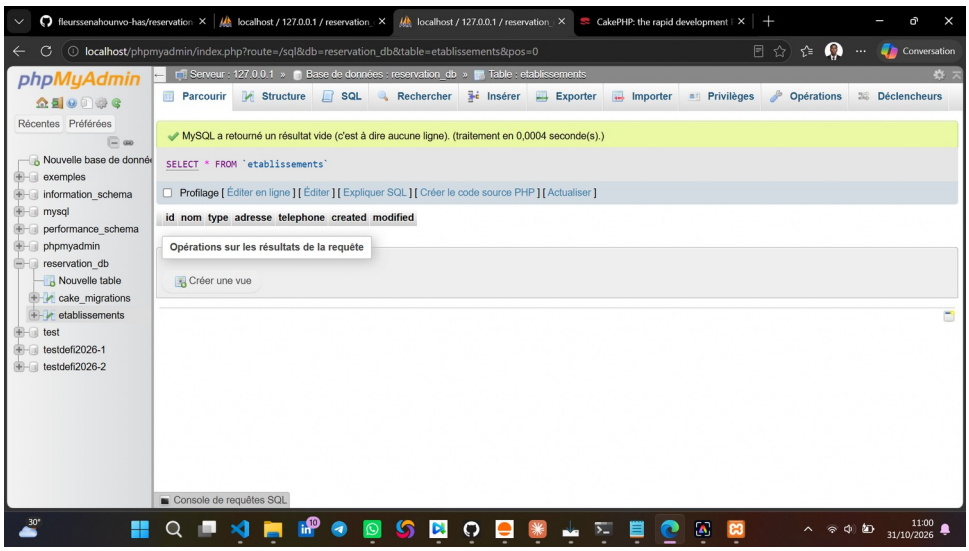


Figure 8 : colonnes de etablissements (id, nom, type, adresse, telephone, created, modified)

7.2 Table ressources (avec clé étrangère)

Génération

```
bin\cake bake migration CreateRessources etablissement_id:integer nom:string[150]
type:string[30] capacite:integer prix:decimal[10,2]
```

config/Migrations/..._CreateRessources.php (méthode change)

```
public function change(): void
{
    $table = $this->table('ressources');
    $table->addColumn('etablissement_id', 'integer', [
        'null' => false,
    ]);
    $table->addColumn('nom', 'string', ['default' => null, 'limit' => 150, 'null' =>
false]);
    $table->addColumn('type', 'string', ['default' => null, 'limit' => 30, 'null' =>
false]);
    $table->addColumn('capacite', 'integer', ['default' => null, 'limit' => 11, 'null'
=> false]);
    $table->addColumn('prix', 'decimal', [
        'default' => null, 'null' => true, 'precision' => 10, 'scale' => 2,
    ]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addForeignKey('etablissement_id', 'etablissements', 'id', [
        'delete' => 'CASCADE',
        'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

Supprimer un établissement supprime donc aussi ses ressources (`CASCADE`). Le prix est facultatif : une salle peut être sur devis. La structure obtenue dans phpMyAdmin, avec l'icône de clé étrangère sur `etablissement_id` :

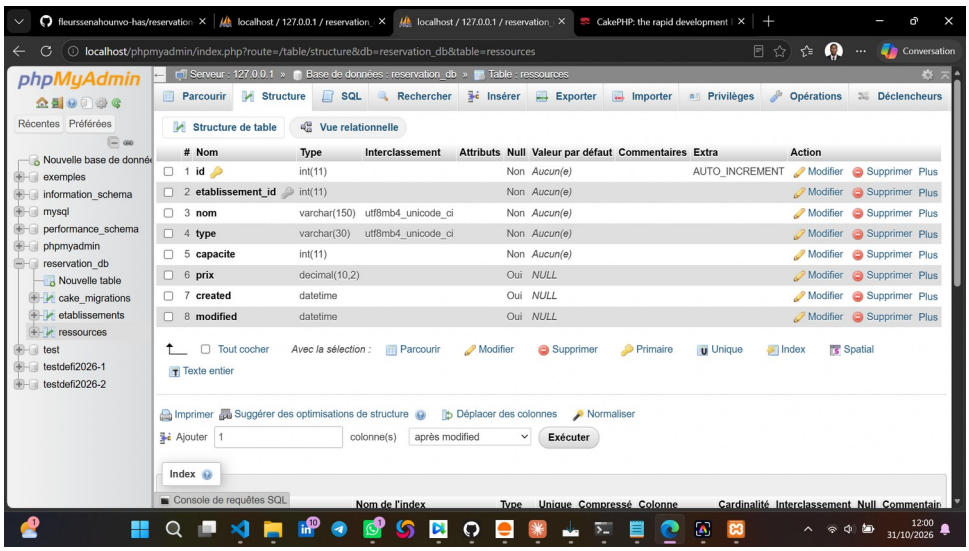


Figure 9 : structure de la table ressources

7.3 Table utilisateurs

Génération

```
bin\cake bake migration CreateUtilisateurs nom:string[100] email:string[150]
mot_de_passe:string[255] role:string[20]
```

config/Migrations/..._CreateUtilisateurs.php (méthode change)

```
public function change(): void
{
    $table = $this->table('utilisateurs');
    $table->addColumn('nom', 'string', ['default' => null, 'limit' => 100, 'null' =>
false]);
    $table->addColumn('email', 'string', ['default' => null, 'limit' => 150, 'null' =>
false]);
    $table->addColumn('mot_de_passe', 'string', ['default' => null, 'limit' => 255,
'null' => false]);
    $table->addColumn('role', 'string', ['default' => 'client', 'limit' => 20, 'null'
=> false]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['email'], ['unique' => true]);
    $table->create();
}
```

L'index unique sur `email` interdit deux comptes avec la même adresse. La colonne `mot_de_passe` fait 255 caractères car elle stockera un mot de passe haché, jamais en clair. Rôles prévus : `client` (par défaut), `gestionnaire`, `administrateur`.

7.4 Table reservations

Décision de conception : aucun compte n'est exigé pour réserver. `utilisateur_id` est donc facultatif et les coordonnées du client sont stockées dans la réservation elle-même.

Génération

```
bin\cake bake migration CreateReservations ressource_id:integer utilisateur_id:integer
debut:datetime fin:datetime statut:string[20] source:string[20]
```

config/Migrations/..._CreateReservations.php (méthode change)

```
public function change(): void
```

```
{
    $table = $this->table('reservations');
    $table->addColumn('ressource_id', 'integer', ['null' => false]);
    $table->addColumn('utilisateur_id', 'integer', ['null' => true]);
    $table->addColumn('nom_client', 'string', ['limit' => 100, 'null' => true]);
    $table->addColumn('telephone_client', 'string', ['limit' => 30, 'null' => true]);
    $table->addColumn('email_client', 'string', ['limit' => 150, 'null' => true]);
    $table->addColumn('debut', 'datetime', ['null' => false]);
    $table->addColumn('fin', 'datetime', ['null' => false]);
    $table->addColumn('statut', 'string', ['default' => 'en_attente', 'limit' => 20,
'null' => false]);
    $table->addColumn('source', 'string', ['default' => 'en_ligne', 'limit' => 20,
'null' => false]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['ressource_id', 'debut', 'fin']);
    $table->addForeignKey('ressource_id', 'ressources', 'id', [
        'delete' => 'CASCADE', 'update' => 'NO_ACTION',
    ]);
    $table->addForeignKey('utilisateur_id', 'utilisateurs', 'id', [
        'delete' => 'SET_NULL', 'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

L'index composé sur `(ressource_id, debut, fin)` garde rapide la recherche de chevauchements. Si un utilisateur est supprimé, ses réservations restent (`SET_NULL`).

7.5 Table blocages

Génération

```
bin\cake bake migration CreateBlocages ressource_id:integer debut:datetime fin:datetime
motif:string[255]
```

config/Migrations/..._CreateBlocages.php (méthode change)

```
public function change(): void
{
    $table = $this->table('blocages');
    $table->addColumn('ressource_id', 'integer', ['null' => false]);
    $table->addColumn('debut', 'datetime', ['null' => false]);
    $table->addColumn('fin', 'datetime', ['null' => false]);
    $table->addColumn('motif', 'string', ['limit' => 255, 'null' => true]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['ressource_id', 'debut', 'fin']);
    $table->addForeignKey('ressource_id', 'ressources', 'id', [
        'delete' => 'CASCADE', 'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

7.6 État final des migrations

bin\cake migrations status

```
+--------+----------------+----------------------+
| Status | Migration ID    | Migration Name       |
+--------+----------------+----------------------+
| up     | 20261031102743  | CreateEtablissements |
```

```
| up     | 20261031105753  | CreateRessources     |
| up     | 20261031120440  | CreateUtilisateurs   |
| up     | 20261031123359  | CreateReservations   |
| up     | 20261031124210  | CreateBlocages       |
+--------+-----------------+----------------------+
```

7.7 Modèle de données

Table	Colonnes
etablissements	id, nom, type, adresse, telephone, created, modified
ressources	id, etablissement_id (clé étrangère), nom, type, capacite, prix, created, modified
utilisateurs	id, nom, email (unique), mot_de_passe (haché), role, created, modified
reservations	id, ressource_id (clé étrangère), utilisateur_id (facultatif), nom_client, telephone_client, email_client, debut, fin, statut, source, created, modified
blocages	id, ressource_id (clé étrangère), debut, fin, motif, created, modified

8. Problèmes rencontrés et solutions

Problème	Cause	Solution
ERR_CONNECTION_REFUSED sur localhost:8765	Le serveur n'est pas lancé, ou CakePHP n'est pas installé	Installer le projet, puis `bin\cake server` dans un terminal laissé ouvert
« The zip extension and unzip/7z commands are both missing »	Extension zip désactivée dans php.ini	Activer `extension=zip`, rouvrir le terminal, supprimer le dossier incomplet, relancer l'installation
Extension intl absente de `php -m`	Désactivée dans php.ini	Activer `extension=intl` dans `C:\xampp\php\php.ini`
`Datasources` tapé dans le terminal : commande introuvable	C'est un mot à chercher dans le fichier, pas une commande	Ouvrir `config/app_local.php` et utiliser Ctrl + F dans l'éditeur
Modification faite dans `app_local.example.php`	Mauvais fichier : c'est un modèle	Annuler (`git checkout config/app_local.example.php`) et modifier `app_local.php`
`git push` rejeté (fetch first)	Le dépôt GitHub avait déjà un commit (README, .gitignore)	`git pull origin main --allow-unrelated-histories --no-rebase`, résoudre les conflits, puis push
`git add docs` : pathspec did not match	Le dossier `docs` n'existe pas encore	`mkdir docs`, y copier le fichier, puis `git add docs`
Erreur 1005, errno 150 sur la clé étrangère de `ressources`	`etablissement_id` déclaré non signé (`'signed' => false`) alors que `etablissements.id` est `int(11)` signé	Retirer `'signed' => false` : les deux colonnes doivent avoir exactement le même type
Erreur de syntaxe dans la migration `ressources`	`$table->$table->addColumn(...)` en trop, et	Recoller le fichier complet corrigé

Problème	Cause	Solution
	colonne `etablissement_id` supprimée par erreur	
`migrate` n'affiche pas de ligne « migrated »	Table déjà dans l'état voulu	Vérifier avec `migrations status` et phpMyAdmin

Leçon retenue : avant de déclarer une clé étrangère, vérifier le type exact de la colonne référencée (`SHOW CREATE TABLE nom_table;`). Une erreur 150 signifie presque toujours que les deux colonnes ne sont pas du même type.

9. Bilan de la semaine 1 et suite

9.1 Livrables réalisés

- Cahier des charges : problème, objectifs, périmètre, acteurs.
- 10 user stories et classement des fonctionnalités (indispensables, utiles, bonus).
- Diagramme de cas d'utilisation.
- Environnement installé : PHP 8.2, Composer, CakePHP 5.4, MySQL, Git.
- Dépôt GitHub public avec historique propre et fichiers sensibles exclus.
- Base de données complète : 5 tables liées par clés étrangères, créées par migrations versionnées.

9.2 Prochaines étapes (semaine 2 : conception)

1. Diagramme de séquence de l'opération « réserver un créneau », avec la vérification des conflits.
2. Diagramme de classes : `Etablissement`, `Ressource`, `Reservation`, `Blocage`, `Utilisateur` et `ServiceReservation`.
3. Maquettes des écrans : liste des ressources, calendrier de disponibilité, formulaire de réservation.

Le cœur technique du projet sera la classe `ServiceReservation`. Dans une transaction, elle verrouille la ressource, cherche une réservation active ou un blocage qui chevauche la période demandée, puis insère la réservation ou la refuse avec un message clair.

Annexe : commandes essentielles

Environnement

```
php -v
composer -V
php -m | findstr intl
php --ini
```

Projet

```
cd C:\xampp\htdocs\reservation-app
bin\cake server
```

Migrations

```
bin\cake bake migration NomMigration champ:type
bin\cake migrations migrate
bin\cake migrations status
bin\cake migrations rollback
```

Git

```
git status
git add .
git commit -m "Message"
git push
git pull
```